

OnlineJudge

High Level Design Document

Stack: MongoDB • Express.js • React • Node.js

1. Overview

OnlineJudge is a full-stack competitive programming platform built with the MERN stack. Users can browse problems, write solutions in a browser-based coding environment, and receive automatic verdicts — similar to platforms like LeetCode or Codeforces.

The platform handles code submission securely by isolating execution inside Docker containers, ensuring that user-submitted code cannot affect the host system or consume unbounded resources.

2. Features

- User registration and login with JWT-based authentication
- Browse a list of problems with difficulty tags (Easy / Medium / Hard)
- Problem detail page with a full coding arena — code editor, language selector, run, submit, and test case panels
- Verdict returned: Accepted, Wrong Answer, Time Limit Exceeded, Runtime Error, or Compilation Error
- Global leaderboard ranking users by number of unique problems solved
- Custom Input panel to test code against any user-defined input without affecting the official verdict
- AI Coding Assistant available throughout the platform for conceptual help
- AI Hint Generator that provides targeted hints on failed submissions without revealing solutions
- AI Code Quality Review triggered on Accepted submissions highlighting efficient, improvable, and bottleneck sections

3. Supported Languages

For the initial version, the judge will support the following languages:

Language	Compiler / Runtime	File Extension
C++	g++	.cpp
Java	javac / JVM	.java
Python	Python 3	.py

4. Database Design (MongoDB)

The application uses four MongoDB collections:

4.1 users

Field	Type	Description
_id	ObjectId	Auto-generated primary key
fullName	String	Full name of the user
email	String (unique)	Login email address
passwordHash	String	bcrypt-hashed password
createdAt	Date	Account creation timestamp

4.2 problems

Field	Type	Description
_id	ObjectId	Auto-generated primary key
title	String	Problem name shown on listing
statement	String	Full problem description
difficulty	Enum	Easy Medium Hard
tags	String[]	Topic tags e.g. Arrays, DP
timeLimit	Number	Execution time limit in seconds
memoryLimit	Number	Memory limit in MB

4.3 testCases

Field	Type	Description
_id	ObjectId	Auto-generated primary key
problemId	ObjectId (ref)	Reference to problems collection
input	String	Test case standard input
expectedOutput	String	Expected standard output
isSample	Boolean	Shown to user if true, hidden otherwise

4.4 submissions

Field	Type	Description
_id	ObjectId	Auto-generated primary key
userId	ObjectId (ref)	Reference to users collection
problemId	ObjectId (ref)	Reference to problems collection
code	String	Raw source code submitted
language	Enum	cpp java python
verdict	Enum	Accepted Wrong Answer TLE CE RE
submittedAt	Date	Auto-set submission timestamp

5. API Routes (Express.js)

5.1 Auth Routes — /api/auth

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive a JWT

5.2 Problem Routes — /api/problems

Method	Endpoint	Description
GET	/api/problems	Fetch all problems (list view)
GET	/api/problems/:id	Fetch a single problem + sample test cases

5.3 Submission Routes — /api/submissions

Method	Endpoint	Description
POST	/api/submissions	Submit code for evaluation (auth required)
POST	/api/submissions/run	Run code against custom input, no verdict stored
GET	/api/submissions/leaderboard	Fetch top users ranked by problems solved

5.4 AI Routes — /api/ai

Method	Endpoint	Description
POST	/api/ai/chat	Send a message to the AI coding assistant
POST	/api/ai/hint	Get a hint for a failed submission (auth required)
POST	/api/ai/review	Get AI code quality review for an Accepted submission

5.5 API Error Responses

All endpoints follow a consistent JSON error format: { error: string, message: string }. The table below lists the standard HTTP status codes returned across the API:

Status Code	Meaning	When It Is Returned
200 OK	Success	Request processed successfully
201 Created	Resource created	New user registered or submission stored
400 Bad Request	Invalid input	Missing fields, unsupported language, malformed request body
401 Unauthorized	Auth required	JWT missing or expired on a protected route
403 Forbidden	Access denied	Authenticated user lacks permission for the resource
404 Not Found	Resource missing	Problem ID or submission ID does not exist
429 Too Many Requests	Rate limited	User has exceeded submission rate limits
500 Internal Error	Server error	Unhandled exception, worker crash, or DB failure

6. Frontend Screens (React)

Screen 1 — Home / Problem List

- Displays a table of all problems with: title, difficulty badge (colour-coded), and tags
- Each row links to the individual problem page
- Header has Login / Register buttons

Screen 2 — Problem / Coding Arena

- Left panel: problem statement, constraints, and sample test cases
- Right panel: code editor, language dropdown (C++ / Java / Python)
- Run button: executes against sample test cases and shows output inline
- Submit button: queues for full evaluation against all hidden test cases
- Verdict panel: shows result (Accepted, Wrong Answer, TLE, etc.) after evaluation completes
- Submissions tab (inside the statement panel): shows the user's past submission history for that problem with verdict, language, and timestamp
- Custom Input panel: user can type their own input and hit Run to test against it — output is shown immediately without affecting the official verdict

Screen 3 — Leaderboard

- Ranks users by number of problems solved (Accepted submissions only)
- Columns: Rank, Username, Problems Solved, Languages Used
- Ties broken by earliest date of last accepted submission

7. Evaluation System

7.1 Execution Flow

1. User submits code via the frontend
2. Backend receives the submission and places a job on a message queue (e.g. Bull + Redis)
3. A worker picks up the job asynchronously
4. Worker spins up a Docker container with the correct language image

5. Submitted code is written to a temp file inside the container
6. Container compiles (if needed) and runs the code against each test case input
7. Output is compared to expectedOutput for every test case
8. Verdict is saved to the submissions collection and returned to the frontend

7.2 Docker Sandboxing

- Each submission runs in an isolated container — no shared filesystem with the host
- Container is assigned a hard memory cap (e.g. 256 MB) to prevent memory abuse
- CPU quota capped per container (e.g. 50% of one core) to prevent CPU hogging by a single submission
- Execution is killed after the problem's timeLimit to enforce TLE
- Network access is disabled inside containers
- Container filesystem is read-only except for the designated temp file directory
- Containers are force-killed and removed immediately after evaluation completes, regardless of outcome

7.3 Fault Tolerance & Retries

The message queue handles job failures gracefully to ensure no submission is silently lost:

- Bull automatically retries a failed job up to 3 times before marking it as permanently failed
- If a worker crashes mid-execution, Bull detects the stalled job and re-queues it for another worker to pick up
- Jobs that exhaust all retry attempts are moved to a dead-letter queue for manual inspection and debugging
- The frontend receives a 'Judging Failed' verdict if all retries are exhausted, and the user is prompted to resubmit

7.4 Scalability

The evaluation pipeline is designed to scale horizontally as submission volume grows:

- Multiple worker instances can run in parallel, all consuming from the same Bull queue — adding workers increases throughput linearly
- Workers are stateless, so they can be deployed across multiple machines without coordination overhead
- Redis acts as the single coordination layer, ensuring each job is picked up by exactly one worker with no duplication
- The Express.js API layer can be scaled horizontally behind a load balancer (e.g. Nginx) as request volume increases
- MongoDB supports replica sets for read scaling and high availability

8. Challenges & Solutions

Challenge	Solution
Thundering Herd — many simultaneous submissions	Message queue (Bull + Redis) buffers submissions; workers process them asynchronously
Malicious code (infinite loops, fork bombs, file access)	Docker container with memory cap, time limit, and no network access
Unauthorized manipulation of verdicts	JWT auth on all write routes; verdict written only by the worker, never by the client

9. Custom Input / Output

Users can test their code against any input they write themselves, without going through the official evaluation pipeline. This is useful for debugging edge cases not covered by the sample test cases.

9.1 How It Works

1. User types custom input into the Custom Input panel in the coding arena
2. Clicking Run sends the code + custom input to `POST /api/submissions/run`
3. The backend spins up a Docker container and pipes the custom input to the program's stdin
4. stdout and stderr are captured and returned directly to the frontend
5. No verdict is generated and nothing is stored in the database

9.2 Key Constraints

- Custom runs are subject to the same Docker sandboxing rules as official submissions (memory cap, time limit, no network)
- Output is truncated beyond a reasonable size limit to prevent output flooding
- Custom runs do not count toward submission history or leaderboard ranking

10. AI Integration

The platform integrates a dedicated AI Service powered by a Large Language Model (LLM). Unlike the deterministic code evaluation pipeline, the AI service is optional and focuses solely on improving the user's learning experience. It analyzes the current coding context — including the problem statement, user code, verdict, compiler errors, and execution results — to provide intelligent assistance without affecting official judging results.

10.1 AI Coding Assistant

A global AI assistant is available throughout the platform. Users can ask conceptual questions, request algorithm explanations, clarify problem statements, or seek guidance on programming concepts without leaving the coding environment.

Example queries:

- Explain Binary Search
- What is the difference between BFS and DFS?
- How do I optimize nested loops?
- Explain this problem in simpler terms

10.2 AI Hint Generator

When a submission results in a Compilation Error, Runtime Error, Wrong Answer, or Time Limit Exceeded, a Get Hint button appears. Instead of revealing the complete solution, the AI analyzes the current submission context and generates a concise hint that guides the user toward the mistake while preserving the learning experience.

Context sent to the AI:

- Problem statement
- User's source code and programming language
- Compiler / runtime error messages
- Current verdict and failed test case info (where applicable)

The AI intentionally avoids generating complete solutions or revealing hidden test cases.

10.3 AI Code Quality Review

When a submission is Accepted, users receive an optional AI-generated quality review alongside the verdict. The AI evaluates the solution for readability, efficiency, maintainability, and potential optimizations, and analyses the code using three visual indicators:

-  Efficient — well-optimized implementation
-  Can Improve — correct but with opportunities for simplification or optimization
-  Potential Bottleneck — sections that may impact performance or readability under larger constraints

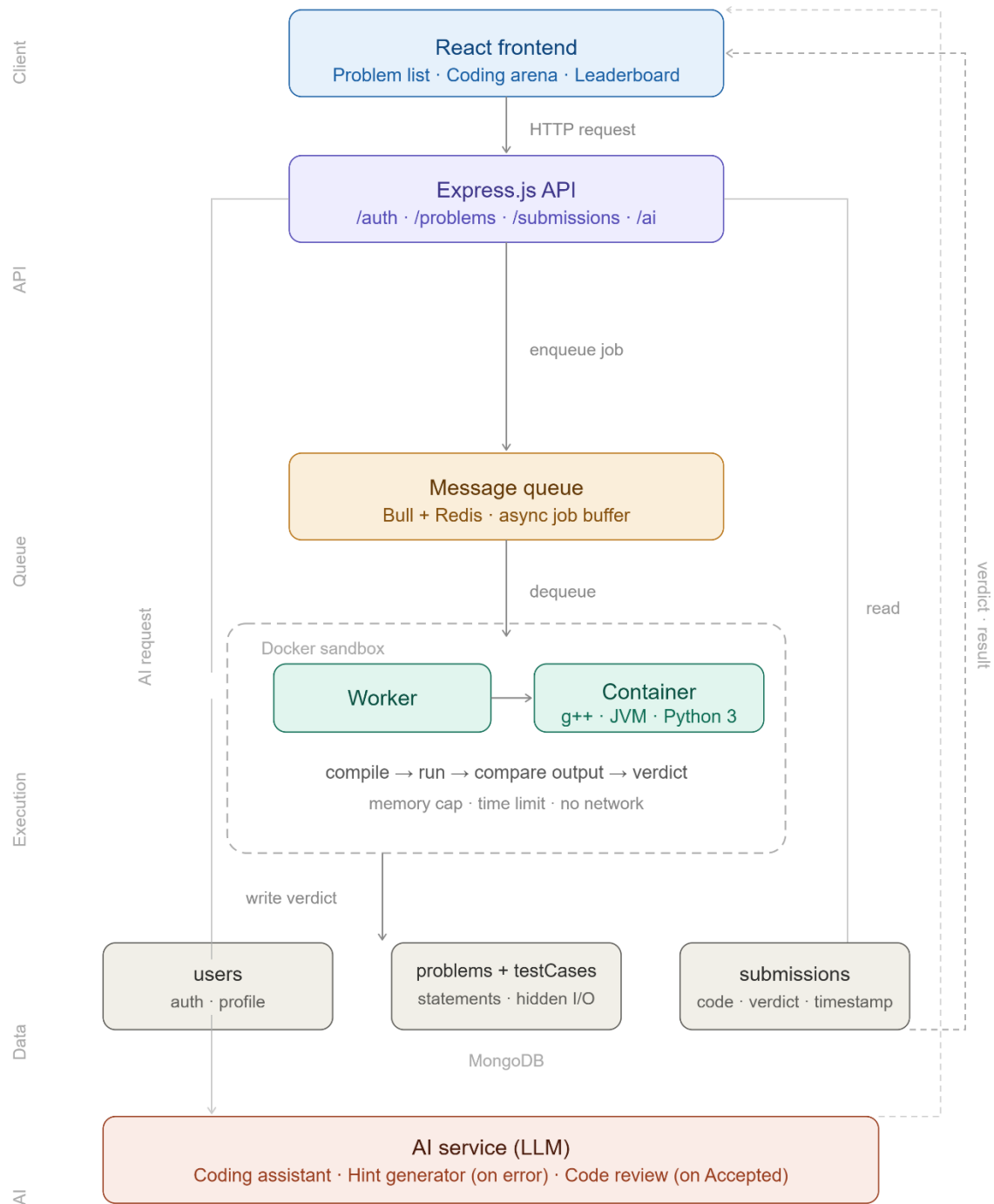
10.4 AI Context Flow

Feature	Triggered When	Context Passed to LLM
AI Coding Assistant	Anytime	User message + problem statement
AI Hint Generator	On CE / RE / WA / TLE	Code, verdict, errors, test results
AI Code Quality Review	On Accepted	Accepted code + problem statement

11. Tech Stack Summary

Layer	Technology
Frontend	React
Backend	Node.js, Express.js
Database	MongoDB (via Mongoose)
Queue	Bull + Redis
Code Execution	Docker containers (g++, JVM, Python 3 images)
Auth	JWT (JSON Web Tokens) + bcrypt
AI Service	LLM API (e.g. OpenAI / Anthropic)

12. Architecture Diagram



By: Aruveer Singh